

INTER IIT TECH MEET

Problem Statements

Four problems, one theme: make a machine understand its situation and act on it, under constraints that do not go away when the demo is over.

FOR STUDENT TEAMS • READ ALL RULES BEFORE YOU START BUILDING

CONTENTS

- 01** **Drone Fleet Management for Timed Deliveries** **HARD** **PAGE 4**

Systems & optimization — ten drones, one server, deadlines to meet, batteries to watch and not enough charging pads.
- 02** **Semantic Mapping of an Indoor Environment** **HARD** **PAGE 8**

Perception — detect objects from an RGB-D stream, store their coordinates, and do it on edge hardware with no pre-built map.
- 03** **Kinodynamic Planning for an Ackermann Vehicle** **MEDIUM** **PAGE 11**

Planning & control — drive a car that cannot move sideways into a parking slot, over a TCP interface, in C++.
- 04** **Hardware Abstraction Driver: CAN and UART** **MEDIUM** **PAGE 14**

Embedded & systems — write the layer between ROS 2 and the electronics: raw CAN frames out, encoder and IMU telemetry in, nothing blocking.

How to Approach These Problems

Every problem in this booklet can be made to look solved. A framework will run, a demo will play, and nothing will have been understood. That is not what we are looking for, and it is usually obvious within the first two minutes of questions.

WHAT WE ARE NOT TESTING

Whether you can install ROS 2, get a simulator running, or wire a library into a project. Those are setup tasks. They take time, they are worth learning, and they carry almost no marks. A team that spends three weeks fighting a toolchain and one week on the actual problem has answered the wrong question.

WHAT WE ARE TESTING

- **How many edge cases you can find.** The happy path is the easy quarter of every one of these problems. What happens when the sensor returns nothing, the deadline cannot be met, the device disconnects mid-command, or two things arrive at once — that is where the work is, and where the marks are. Bring us the list of cases you thought of. Finding one we did not list is worth more than handling five we did.
- **How well you understand what you built.** Every line in your repository is yours to defend. Why this value and not a larger one. What breaks if this assumption fails. Where your design is still unsafe. “It works” is not an explanation.
- **How you approached the problem.** What you tried first, why it failed, what you changed, and what you deliberately chose not to do. The path matters as much as the destination — we would rather read an honest account of three attempts than a clean write-up of one.

A SENSIBLE ORDER OF WORK

Get the simplest possible version running end to end before you make any part of it good. A slow, ugly, complete pipeline gives you a baseline to measure against and tells you early which part is actually hard. Optimising a component before the system runs is time spent on the wrong thing. Then attack your own system: break it deliberately, write down what happened, and fix what matters.

Where a problem leaves a choice open — an algorithm, a model, a bus, a reference frame — that is deliberate. We are not withholding an answer. We want to see which one you pick, what you compared it against, and whether you can defend it.

USING LLMs

LLMs and AI coding tools are allowed on every problem here. We are not going to pretend they do not exist, and we are not testing whether you can type faster than an autocomplete. Use them.

One condition: **you own everything you submit.** If it is in your repository, you know why it is there and what happens if it changes. Expect questions on any line, without warning — why that threshold, why that data structure, what your code does when this input is malformed. “The AI wrote that part” is not an answer, and it becomes clear almost immediately. A team that built something simpler and can defend every line of it will beat a team with a better demo they cannot explain.

HOW YOU SUBMIT

One GitHub repository link, and one demo video. There are no slides. The video is how you present your work, so edit it properly: show the system actually running, cut the dead time, caption what we are looking at, and leave a failure case in. A raw unedited screen recording with no explanation does not count as a submission. Everything else a problem asks for — notes, logs, results — goes in the repository.

Drone Fleet Management for Timed Deliveries

Ten drones, one central server, and a stream of packages that each have to reach a location before a deadline. Deciding *which drone goes where, and when* is the whole problem — and it has to hold up when batteries age and every charging pad is busy.

FLEET	ARCHITECTURE	EACH PACKAGE	CHARGING	SIMULATOR
10 drones	One server, drones as clients	Location, weight, deadline	Limited pads, shared	Your choice

1 The Problem

A delivery company has ten drones and a lot of packages. Each package has a weight, a drop location and a time it must arrive by. A central server tracks every drone and decides who flies where.

One delivery is easy. Ten drones, a queue of orders, batteries that drain and slowly wear out, and a handful of shared charging pads is a different problem — and it is the one you are solving. Send the wrong drone and you burn range you needed later. Wait too long for a better option and you miss the deadline. Send everything at once and the whole fleet arrives at the pads together, and nothing flies for the next forty minutes.

2 What You Build

A central server. It holds fleet state, accepts delivery requests, and assigns them. Every decision about who flies where is made here.

Drone clients. Each drone connects to the server, reports position, battery and payload, accepts the jobs it is given, and flies them.

A fleet portal. A live view: where each drone is, what it carries, what it has been assigned, battery and health, which deliveries are on time and which are at risk, and which pads are occupied.

Portal

A live view is not decoration. It is how you show your assignment logic was sensible.

3 The Fleet You Are Given

Start from these numbers. They define the problem. If you change one, say so and justify it.

PARAMETER	VALUE	NOTE
Drones	10	Identical to begin with. See section 7.
Max payload	2.5 kg	A drone cannot lift more. Package weights run 0.2–2.5 kg.
Cruise speed	12 m/s	Falls as payload rises — model it.
Endurance	~25 min at full charge	At full payload. Longer when flying light or empty.
Charge time	~40 min, empty to full	A drone on a pad is out of the fleet.
Battery degradation	–0.05% usable capacity per full cycle	Permanent. A drone at cycle 400 no longer has its day-one range.
Base station	1	All drones launch from and return to it. See section 7 for the multi-base extension.
Charging pads	3	Three pads for ten drones.
Deadlines	10–45 min from request	Requests arrive continuously, not in a batch.

Payload matters twice over: a drone cannot carry more than its limit, and a heavy package drains the battery faster and slows the drone down — so a job that fits on paper can still be the wrong job for that drone.

4 The Decision That Matters

A new request arrives. Somewhere in your fleet a drone is already in the air, heading roughly that way. What do you do?

```
new request → 1.9 kg to drop point D, deadline in 14 min

drone 3  airborne, passes near D in 5 min, battery 41%, already carrying 1.2 kg
drone 7  idle at base, 9 min to D, battery 100%, capacity down 4% from age
drone 2  charging, ready in 6 min, then 7 min to D

assign to whom? or hold the request and wait?
```

Every option costs something. Diverting the airborne drone saves a takeoff but risks its current delivery, and it may not have the spare payload. The idle drone is safe but burns a full charge on one package. Waiting for the charging drone might make the deadline and save both — or miss it by thirty seconds.

Your server makes that call, defensibly, every time. It is balancing at least four things:

- **Deadlines.** A late delivery is a failure, not a small penalty.
- **Energy and payload.** Battery spent is battery unavailable later, and weight makes every leg cost more.
- **Waiting.** Holding a request for a better assignment is allowed and sometimes correct — but a request held too long can no longer be completed at all.
- **Balance across the fleet.** Work should be spread sensibly. A schedule that runs three drones into the ground while seven sit idle looks fine on delivery numbers and quietly wears out a third of your fleet.

We are not prescribing an algorithm. Choosing one, and being able to say why it beats the alternatives you tried, is the task. What we do require is that your server can explain any single decision afterwards: why this drone, at this moment, and not the others.

5 Charging

Three pads, ten drones. A pad serves one drone at a time, and a drone on a pad is out of the fleet until it comes off.

Questions your design has to answer: who gets a pad when several drones are low at once? Does a drone reserve one before it starts flying home, and what happens to that reservation if it is diverted or arrives late? Do you send a drone to charge at 30% because the pads are free now, or keep it working and risk finding all three busy at 12%?

THE RULE THAT MATTERS

A drone must never be assigned a job it cannot finish, including the flight back to a pad, using its *current degraded* capacity rather than its nameplate figure. Running out of battery in the air is a crash, and a fleet manager that allows one has failed regardless of its delivery numbers.

6 Edge Cases

A large part of the marks. A system that works when everything goes right is not a fleet manager. We want the cases you thought about, what your system does in each, and how you tested it. These we already expect you to have covered — they are not the full list:

- A deadline no drone can meet. Does the server accept the request and fail, or reject it up front?
- A package heavier than any available drone can carry.
- Every pad is occupied and a drone needs one now.
- An urgent request arrives with all ten drones committed.
- Two requests, one drone that could serve either, and both deadlines cannot be met.
- A drone is diverted after it had already reserved a pad.
- A delivery is reassigned mid-flight while the first drone is still carrying the package.
- A drone reports a battery level or position that cannot be true.
- Wind or a slow leg turns an on-time delivery into a late one. When do you notice, and what then?

Finding cases we have not listed scores higher than handling the ones we have.

7 Extensions BONUS

Three optional directions, in rough order of value. Attempt them *only* once the base system is stable — a broken extension on a weak base scores lower than a solid base alone.

A — A FLEET OF DIFFERENT DRONES

Real fleets are not ten identical airframes. Make them differ — a heavy-lift drone that carries 5 kg but flies slowly and drains fast, a light fast one capped at 1 kg, an older unit with 20% of its capacity already worn away. The question sharpens from *which drone is free* to *which drone is right for this job*: the fast one cannot lift a 4 kg package, and sending your only heavy-lift drone across town with 300 g leaves the next heavy package with nothing to fly on. Define your parameter set, state it, and show your logic choosing sensibly under it.

B — COMMUNICATION FAILURES AND RECOVERY

Links drop. A drone goes quiet mid-flight and the server must decide what that means without being able to ask. Both sides need defined behaviour. On the drone: how long does it keep flying its last instruction before deciding it is on its own, and what then — continue, hold, or return — while staying inside the range that gets it home. On the server: when is a silent drone declared lost, what happens to its package, and how do you avoid giving the same delivery to two drones?

One rule if you attempt this: a drone that reconnects must **not** resume its old command. It re-identifies, reports position and payload, and waits for a fresh assignment. Think hardest about the drone you declared lost, whose package you reassigned, reconnecting while still carrying it.

C — MULTIPLE BASE STATIONS

Replace the single base with three, two pads each. The charging question gains a second half: not just who gets a pad, but *which base* they fly to. The nearest may be full while a further one sits empty, and the extra leg costs range the drone may not have.

8 Simulation

Use whatever you like — PX4 or ArduPilot SITL, Pixhawk, Aerostack, Gazebo, or any framework you are comfortable with. We are marking the fleet logic, not the flight stack.

IF THE SIMULATOR FIGHTS YOU, WALK AWAY FROM IT

If you cannot get a drone simulator running — and with ten of them, that happens — build your own 2D simulation with OpenCV and show the fleet moving on a map. That is completely acceptable and costs you nothing. A clean OpenCV visualisation with sound assignment logic beats a broken SITL setup with no logic behind it. Do not spend three weeks on the simulator and one week on the actual problem.

9 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	One link. Server, drone clients and portal, with a README that lets us clone it and run the whole thing.
Demo video	An edited recording of a full run — requests arriving, drones assigned, batteries draining, pads filling — including a run that goes wrong.
Algorithm note	Your assignment and charging algorithms, why you chose them over what else you tried, and where they break down.
Edge case document	Every case you identified, what your system does, and how you tested it.
Results	Deliveries on time and missed, distance and energy used, idle time, pad utilisation, and how evenly work was spread.

10 What We Judge On

Assignment quality	Deadlines met, energy and payload used sensibly, and whether a specific decision holds up when we ask about it.
Edge case coverage	

	How many you found, and whether the system genuinely handles them rather than listing them.
Charging strategy	Pads used sensibly under contention, and no drone ever stranded.
Balance	Work spread across the fleet rather than concentrated on a few airframes.
Portal	Does it show enough to understand what the fleet is doing, and why.
Extensions (<i>bonus</i>)	Any of section 7 done well, on top of a base system that already works.

The first two carry the most weight. A fleet that delivers slightly less efficiently but never strands a drone and never accepts an impossible job beats a faster one that falls apart the first time something unexpected happens.

QUESTIONS YOU SHOULD EXPECT

Why your server sent drone 4 and not drone 9 at 03:12 in your own demo. What your cost function actually weighs, and what changes if you double one term. What happens if two requests arrive in the same millisecond. Why your charging threshold is that number, and whether it changes as a battery ages. What your system does when a drone reports a battery level that cannot be true.

Semantic Mapping of an Indoor Environment

Build a vision pipeline that detects objects in a house, works out where each one is, and stores those coordinates. It has to run on edge hardware, and it cannot use a pre-built map.

SIMULATOR	ROBOT	SENSOR	TARGET HARDWARE	MEMORY
Gazebo — small house world	TurtleBot 3 or 4	RGB-D camera	Jetson Nano / Raspberry Pi	8 GB peak

1 The Problem

A robot with a depth camera is placed in a house it has no map of. It can see the room, but it does not know that the large flat thing in front of it is a bed, and it has no way to come back to that bed later.

People give robots instructions in terms of objects, not numbers — “go to the sofa”, not “go to 2.4, 1.1”. For the robot to act on that, it needs a stored list of objects and their positions. That list is a **semantic map**, and building it is your task.

2 What You Build

A vision pipeline that runs on a TurtleBot in the Gazebo small house world. It takes the RGB and depth stream as input. It outputs a stored list of objects.

```
{ "id": 7, "label": "coffee_table",
  "position": [x, y, z], // base point, in your chosen world frame
  "confidence": 0.87, "extent": [w, d, h] // extent optional }
```

Three requirements:

Segment, do not just box. You need the object's boundary, not a rectangle around it. A bounding box includes background pixels, and those corrupt the depth values you read from it. A pixel mask gives you only the depth that belongs to the object.

Label the object. Bed, sofa, coffee table, dustbin, chair, TV, fridge, door, and so on. Cover as many classes as your pipeline handles reliably. More classes is better, but wrong labels are worse than fewer labels.

Store its position. Convert the object's pixels into one 3D coordinate in a fixed world frame and save it. Use the **base point** — where the object meets the floor — because that is the point a navigation stack can drive to.

WHAT WORKING LOOKS LIKE

After a run, one command prints your stored map: “11 objects — sofa (1.2, 3.4), dustbin (-0.8, 2.1), bed (4.5, 0.9)...”. Send the robot to any entry in that list and it arrives at the right place.

3 Rules

NON-NEGOTIABLE

- **No pre-built map.** You cannot load a map saved from an earlier run, and you cannot read object positions out of the Gazebo world file. The robot starts with nothing, every run.
- **Peak RAM stays under 8 GB** for the whole pipeline. Measure it and report it.
- **It must run on Jetson Nano or Raspberry Pi class hardware.** Develop on your laptop, but every model you pick must be one that can actually be deployed on that board. A pipeline that only runs on a desktop GPU does not count, however accurate it is.
- **You must be able to explain everything you submit.** See *Read This First* at the front of this booklet.

The first rule is the core of the problem. Knowing where things are in advance makes this easy. Building that reference from nothing is the actual work.

Why masks
A box around a bed also contains floor, wall and half a nightstand.

Base point
The centre of a tall shelf sits in mid-air. A robot cannot drive to it.

4 The Coordinate Problem

Your camera gives you a position in the **camera frame**: “dustbin, 1.8 m ahead, 0.3 m to the left”. The robot then moves, and that number no longer points at the dustbin. Camera-frame coordinates cannot be stored.

You need a fixed reference frame that holds for the whole run. **We are not telling you what it should be.** Picking it, building it and defending it is part of the problem.

QUESTIONS YOUR DESIGN HAS TO ANSWER

- **Where is your origin?** The start pose is one option. Is it enough? What breaks if the robot is moved and restarted somewhere else?
- **How do you track the robot's pose?** Whatever you choose, say what it costs you in compute and how much it drifts.
- **What happens as drift builds up?** After a full loop of the house, is the sofa still where you recorded it? How far off is it? Do you correct it, or measure and report the error?
- **What if you see the same object again?** You will pass the same sofa four times. Do you end up with four sofas in your map?
- **Which single point represents the object?** You have thousands of depth values inside a mask. How do you reduce them to one coordinate, and then to a point on the floor?

There is no single correct answer. A simple approach you fully understand, whose failure cases you can name, scores better than a complex one you copied.

5 Running on Small Hardware

The constraint is the point of this problem, and a large share of the marks sits here. Wiring a large model to a camera is easy. Making it small and fast while keeping it correct is the work.

We prescribe no models, no runtimes and no techniques. Finding them is your job, and it is left open on purpose — two teams that solve this well should not end up with the same pipeline. Whatever you pick, be ready to say why you picked it over what else you tried, and what it cost you.

MEASURE AND REPORT

Peak RAM · end-to-end FPS and per-stage latency · model size on disk · accuracy. Keep a log from day one: every change you made, the numbers before and after, and one line on whether you kept it. A row reading “4 FPS → 19 FPS, mask IoU -3%” tells us more than any final number, and that log is the strongest thing you can show a judge.

6 Optional Extension **BONUS**

You may use a VLM or VLA. Stopping at a clean, fast, accurate map is a complete submission. If you want to go further: pass your stored map and a plain instruction to the model, and have it return a structured command the robot can execute. Send that to the navigation stack and let the robot drive there.

```
Input: "go and check the dustbin" + your stored object map
Output: { "action": "navigate_to", "target": "dustbin",
          "coordinates": [-0.82, 2.14, 0.0] }
```

The output must be parseable structure, not prose — validate it, and handle the case where the model returns nonsense. Respect the same compute budget if you claim it runs on the edge; if you offload it, say so. Do not weaken the core pipeline for this. A broken bonus on a weak base scores lower than a strong base alone.

7 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	One link. The working pipeline, and a README that lets us clone it and reproduce your run from scratch.
Semantic map output	A JSON or YAML object list — label, 3D coordinate, confidence — from a real run.
Demo video	An edited recording of the robot exploring, with detections and the map building live, plus a failure case. See the note below.

Benchmark report	The measurements and the change log from section 5.
Design note	A few pages: your reference frame and why, your model choices and why, and the failure modes you know about.

8 What We Judge On

CRITERION

Optimization & edge readiness	Measured gains, real deployability on the target board, memory discipline, and a clear record of your trade-offs.
Pipeline design	Clean, justified architecture. A sound reference frame. Repeat detections handled properly.
Mapping accuracy	Correct labels and coordinates, sensible base points, class coverage, no duplicate objects.
Robustness	Survives a different start pose, route, lighting and occlusion. Degrades instead of failing outright.
Understanding & defence	Can you explain every decision under questioning.
Optional extension	Instruction to structured command, grounded in your own map, ending in real robot motion.

The first two carry the most weight. A team at 92% accuracy and 2 FPS on a desktop GPU loses to a team at 84% and 15 FPS on a Nano. That ordering is deliberate.

QUESTIONS YOU SHOULD EXPECT

Why this model and not the other three you tried. Why this threshold and not a higher one, and what changed when you moved it. Why you reduced the mask that way. What your code does when the depth sensor returns zeros. What happens the third time the robot passes the same sofa.

ONE WARNING

The most common way teams lose this: three weeks spent raising accuracy on a laptop GPU, then finding in the last week that none of it fits the target board. Test against your constraints from week one.

Kinodynamic Planning for an Ackermann Vehicle

Write a C++ client that connects to a car simulator, plans a path the car can actually drive, and streams steering and speed commands until it reaches the goal pose without hitting anything.

SIMULATOR	INTERFACE	LANGUAGE	CONTROL RATE	SCENARIOS
C++ kinodynamic sim, 100 × 100 m grid	TCP socket, port 8091	C++17	20 Hz	4

1 The Problem

A car cannot move sideways. To shift one metre to its left it has to drive forward, turn, reverse, and straighten up. This is what makes parking hard, and it is why a path that looks fine on a map is often a path the car physically cannot follow.

You are given a simulator of such a car. It runs the vehicle dynamics, checks collisions, and exposes everything over a TCP socket. Your job is to write the software that drives it: read the map and the goal, plan a path the car can follow, and send the controls that keep it on that path.

2 What You Build

A C++ client. It connects to the simulator, requests the configuration and the occupancy grid, plans a collision-free path from the start pose to the goal pose, and streams control commands at 20 Hz until the goal is reached.

Two halves, and both have to work:

The planner produces a path the vehicle can actually drive — never a turn tighter than the steering limit, never the car body through an obstacle. Position alone is not enough: heading is part of the state, and so is direction of travel. Reversing is allowed, and in the parking scenarios it is required.

The controller follows that path on the real vehicle model, which will not track it perfectly. It corrects the error as it builds, and brings the car to the goal *pose* — position and heading, within tolerance.

Your logic goes in `apps/candidate_template.cpp`.

3 The Vehicle

Motion follows the Ackermann kinematic bicycle model. Your two controls are target speed v and steering angle δ .

PROPERTY	VALUE	WHAT IT MEANS FOR YOU
Wheelbase	2.5 m	Sets how sharply the car turns for a given steering angle.
Max steering angle	$\delta_{\max} = 0.60 \text{ rad}$ ($\approx 34.3^\circ$)	A hard limit. Commands beyond it are not achievable.
Min turning radius	$R_{\min} = L_w / \tan \delta_{\max} \approx 3.7 \text{ m}$	The tightest arc the car can trace. Any planned curve tighter than this is undrivable.
Body size	4.0 m × 1.8 m	Collisions are checked on the oriented bounding box, so heading changes the footprint.

THE CONSEQUENCE

The car sweeps a $4.0 \times 1.8 \text{ m}$ box that rotates as it turns. A path that clears every obstacle as a line of points can still put a corner of the car into a parked vehicle. Check the body, not the centre.

Non-holonomic
Fewer ways to move than there are dimensions to move in. A car has three — x, y, heading — but only two controls.

Goal pose, not goal point
Arriving at the right place facing the wrong way is not a park.

4 Scenarios

Four scenarios, selected by ID when you launch the simulator. Your client should handle all of them without being rewritten for each.

ID	SCENARIO	WHAT IT TESTS
0	Street parallel parking	Parallel park into a tight slot between two parked cars along a curb.
1	Parking lot bay	Park into a perpendicular bay with tight aisles on either side.
2	Slalom track	Navigate an obstacle course. Continuous steering, no stopping to think.
3	Multi-goal navigation	Reach several goal waypoints in sequence.

5 Protocol

The simulator runs a TCP server on port 8091. Four messages, all newline-terminated plain text.

REQUEST THE CONFIGURATION — CLIENT TO SERVER

Send `Q\n` once at startup. You get back the poses, the vehicle parameters and the map.

```
CONFIG <start_x> <start_y> <start_yaw> <goal_x> <goal_y> <goal_yaw>
      <len> <width> <wheelbase> <max_steer> <max_speed> <min_speed>
      <map_w> <map_h> <res> <orig_x> <orig_y> <cols> <rows>
GRID <num_cells> <val_0> <val_1> ... <val_N> // 0 = free, 1 = obstacle
```

STREAM CONTROLS — CLIENT TO SERVER, 20 HZ

```
CTRL <target_speed_m_s> <target_steering_angle_rad>
e.g. CTRL 1.2 -0.25
```

TELEMETRY — SERVER TO CLIENT

```
TELEMETRY <step_count> <time_ms> <x> <y> <yaw> <v> <delta>
          <is_colliding> <is_goal_reached>
```

UPLOAD YOUR TRAJECTORY FOR VISUALISATION — OPTIONAL

```
TRAJ <x1> <y1> <yaw1> <v1>;<x2> <y2> <yaw2> <v2>;...
```

WORTH KNOWING BEFORE YOU START

The grid arrives once, as a flat array — you convert to world coordinates yourself using `res`, `orig_x` and `orig_y`. Telemetry is a stream, so your loop has to keep reading while it keeps sending. Miss the 20 Hz control rate and the car runs on stale commands.

6 Build and Run

You need GCC/G++ 11 or newer, CMake 3.16+, and OpenCV 4 (libopencv-dev).

```
cd ackermann_kinodynamic_sim
mkdir -p build && cd build
cmake .. && make -j4

./simulator_node 8091 0 # port, then scenario ID 0-3

# in a second terminal
./candidate_template 127.0.0.1 8091
```

Rules

NON-NEGOTIABLE

- **Respect the vehicle model.** No teleporting, no sideways motion, no steering beyond $\delta_{\max}$. If the plan needs a radius under 3.7 m, the plan is wrong.
- **No collisions.** Checked on the oriented bounding box of the full car body.
- **Reach the goal pose** — position and heading within tolerance.
- **One client, all four scenarios.** No per-scenario hard-coding, no replaying a recorded control sequence.
- **C++.** Your logic goes in `apps/candidate_template.cpp`.
- **You must be able to explain everything you submit.** See *Read This First* at the front of this booklet.

We prescribe **no planner and no controller**. Which algorithms you use, and how you combine them, is the problem. Whatever you pick, be ready to say why you picked it over what else you tried, and where it breaks.

Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	One link. Your client, building cleanly against the given CMake project, with a README covering how to run it.
Run logs	One per scenario: time to goal, collisions, final pose error, and whether the goal was reached.
Demo video	An edited recording of all four scenarios with your planned trajectory visible via TRAJ, plus a failure case. See the note below.
Design note	A few pages: your planner, your controller, how they connect, and the cases where your approach struggles.

What We Judge On

CRITERION	
Does it park	Goal reached, in pose, without collisions, across all four scenarios.
Planner & controller design	Sound handling of the non-holonomic constraint, clean separation of planning and control, sensible replanning.
Path quality	Smooth, drivable paths. Sensible clearance. Not more reverse manoeuvres than the situation needs.
Robustness	Handles all four scenarios with one codebase, and recovers when tracking drifts instead of failing outright.
Efficiency	Plans fast enough to hold 20 Hz control, and does not stall the loop when it replans.
Understanding & defence	Can you explain every decision under questioning.

QUESTIONS YOU SHOULD EXPECT

Why this planner and not the other one you tried. What your cost function penalises, and what changes if you double one term. Why the car swings wide on the third turn of the slalom. What your controller does when the tracking error grows instead of shrinking. What happens if the goal is placed inside an obstacle.

Hardware Abstraction Driver: CAN and UART

Write the C++ layer between ROS 2 and the vehicle's electronics: wheel commands out as raw CAN bytes, encoder frames back, an IMU stream off a serial port — and nothing that blocks the control loop.

BUSES	DEVICES	LANGUAGE	CONTROL LOOP	IMU STREAM
SocketCAN + POSIX serial	vcan0 · /tmp/ttyV0	C++ (ros2_control)	100–500 Hz	100 Hz

1 The Problem

In simulation, a robot's wheels turn because a plugin says so. On a real vehicle nothing turns until some software has packed a velocity into eight bytes and put them on a CAN bus. Between the high-level controller and the motors sits a layer most people never write. That is the layer you are writing here.

You are given a firmware emulator that behaves like the vehicle's onboard microcontroller. It listens on a virtual CAN bus and a virtual serial port, acts on what you send, and streams telemetry back.

2 What You Build

A custom C++ hardware interface — a `ros2_control SystemInterface` — that sits under the standard controllers and owns both buses.

```
[ ROS 2 controllers ] (diff drive / joint trajectory)
|
[ your hardware interface ] <-- the deliverable
|
+---> SocketCAN vcan0 ----> [ firmware emulator ] ---> rover kinematics
+---> UART /tmp/ttyV0 ---->
```

Five stubs in `apps/candidate_template.cpp`:

FUNCTION	WHAT IT DOES
init()	Open and configure the non-blocking CAN socket on <code>vcan0</code> , and the termios serial port on <code>/tmp/ttyV0</code> .
computeCanChecksum()	8-bit XOR over payload bytes 0–6.
sendMotorVelocity()	Build and write the 8-byte motor command frame.
readEncoderFeedback()	Non-blocking read of encoder frames, checksum verified before use.
readImuTelemetry()	Non-blocking read and parse of NMEA IMU sentences, CRC verified before use.

3 CAN Protocol

Raw SocketCAN — `AF_CAN`, `SOCK_RAW`, `CAN_RAW` — on interface `vcan0`. Four motors, one ID each way.

Commands go out on `0x101–0x104`. **Encoder feedback** comes back on `0x201–0x204`. In both directions the ID's low nibble is the motor number.

BYTE	TYPE	CONTENTS
0	uint8_t	Motor identifier — 1, 2, 3 or 4.
1–4	float32	Target velocity going out, measured velocity coming back. Little-endian.
5–6	uint8_t[2]	Reserved. Send <code>0x00</code> .
7	uint8_t	XOR checksum of bytes 0 through 6.

Endianness

Float32, little-endian, at byte offset 1 — an unaligned offset.

4 UART Protocol

The IMU streams ASCII NMEA sentences at 100 Hz. Port settings are 115200 baud, 8N1, raw mode.

```
$ROVER,IMU,<accel_x>,<accel_y>,<yaw>*<CRC_HEX>\n
example: $ROVER,IMU,0.120,-0.050,1.5708*3E\n
```

The CRC is two hex characters: the 8-bit XOR of every ASCII character between \$ and *, excluding both. For the example above, that is the XOR over ROVER, IMU, 0.120, -0.050, 1.5708.

Note

Your /tmp/ttyV0 and the emulator's /tmp/ttyV1 are the two ends of one virtual pair.

WHERE THIS ONE BITES

A non-blocking read returns whatever bytes happen to be in the buffer. That is rarely one whole sentence — you will get half of one, or one and a half. If you parse only what a single `read()` handed you, you will drop most of the stream and never see why. Hold a buffer across calls, and pull out complete sentences as they form.

5 Real-Time Rules

NON-NEGOTIABLE

- **Nothing blocks.** `read()` and `write()` run inside a 100–500 Hz control loop. Every CAN and serial call must return immediately when there is no data: `O_NONBLOCK` on the socket, `VMIN=0` and `VTIME=0` on the port. One blocking call stalls the entire robot, not just your driver.
- **Verify before you trust.** Check byte 7 on every CAN frame and the CRC on every NMEA sentence *before* the value reaches wheel or sensor state. A corrupt frame is dropped, not used.
- **No sleeping, no busy-waiting, no retry loops** inside the control cycle to make up for a read that came back empty. Empty is a normal outcome.
- **You must be able to explain everything you submit.** See *Read This First* at the front of this booklet.

Everything above is checkable from the outside. The loop either holds its rate or it does not, and bad frames either reach the state or they do not.

6 Test and Verify

```
./setup_vcan_pty.sh # bring up vcan0 and the pty pair

# terminal 1 – the emulated microcontroller
python3 firmware_emulator/rover_firmware_emulator.py

# terminal 2 – build and run your driver
mkdir -p build && cd build && cmake .. && make -j4
./candidate_template vcan0 /tmp/ttyV0
```

When it is right, you get a live log of encoder feedback and IMU acceleration and yaw. When it is nearly right, you get a log that looks fine and drops a quarter of the frames — so count what you receive against what the emulator sent, rather than trusting the output on sight.

7 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	One link. Your completed hardware interface, building cleanly against the given CMake project, with a README covering how to run it.
Demo video	An edited recording showing encoder and IMU data flowing over the virtual buses, plus a failure case. See the note below.
Evidence it works	Your own counts: frames sent, frames received, checksums failed, sentences dropped.
Loop timing	Measured cycle time for <code>read()</code> and <code>write()</code> at your control rate, with the worst case, not just the average.
Design note	A few pages: how you handle partial reads, what you do with a failed checksum, and where your driver would break on real hardware.

What We Judge On

CRITERION

Correctness on the wire	Frames the emulator accepts, checksums computed right in both directions, floats decoded correctly.
Real-time behaviour	Genuinely non-blocking, holds its rate under load, no hidden waits or sleeps in the cycle.
Data integrity	Corrupt frames rejected, partial sentences handled, nothing bad reaching the state.
Robustness	Survives the emulator restarting, a bus going quiet, and malformed input, without crashing or hanging.
Code quality	Clean resource handling, no leaked descriptors, errors handled rather than ignored.
Understanding & defence	Can you explain every decision under questioning.

Correctness and real-time behaviour carry the most weight. A driver that decodes everything perfectly but blocks for two milliseconds on an empty bus has failed the part of this problem that matters.

QUESTIONS YOU SHOULD EXPECT

Why you configured termios the way you did, and what each flag you set actually turns off. How you get a float out of bytes 1–4, and what your method assumes about alignment. What your code does when `read()` returns `EAGAIN`. What happens if a sentence arrives split across two reads, and how you tested that it does. Why your checksum runs over bytes 0–6 and not the whole frame.